

Lecture 5

ESSLLI Course — *Introduction to Homotopy Type Theory / Univalent Foundations*

Tom de Jong

Additional reading

The citations refer to the file `Resources.pdf`. Rijke’s treatment [Rij25, Chapter III] is essentially the same as ours, although the presentation is a little different at times and the proof below was developed independently.

The original type-theoretic encode-decode proof, the ideas of which lie at the heart of the below and Rijke’s proof, is due to Michael (Mike) Shulman, see Section 8.1 of the HoTT Book for this and more.

I also formalized this note, see: <https://cs.bham.ac.uk/~mhe/TypeTopology/SyntheticHomotopyTheory.Circle.FundamentalGroup.html>

Goal

The goal of this note is to introduce the circle S^1 and to prove that its identity type $\text{pt} \equiv_{S^1} \text{pt}$ is equivalent to the type of integers.

If familiar, you should compare the proof here to the typical proof — and the amount of setting up that it takes — in an introductory algebraic topology course that the fundamental group of the circle is given by the group of integers.

Along the way we introduce important concepts like **higher inductive types** and reasoning with **universal properties in the form of equivalences**.

(See also my expository short paper *Formalizing equivalences without tears* for more on the latter.)

1. Some universal properties

Note that $\mathbb{1}$ has the property that X and $\mathbb{1} \rightarrow X$ are equivalent, for any type X , where we map $x : X$ to the constant function $\lambda u : \mathbb{1}.x$. In fact, this characterizes the unit type, or equivalently, all contractible types.

Lemma (Universal property of contractible types). The following are equivalent for a type A :

1. for every type X , the constants map $X \rightarrow (A \rightarrow X)$, given by $x \mapsto \lambda a.x$ is an equivalence;
2. the constants map $A \rightarrow (A \rightarrow A)$ is an equivalence;
3. the type A is contractible.

Proof. (1) $\Rightarrow$ (2) is immediate. We show (2) $\Rightarrow$ (3) and (3) $\Rightarrow$ (1).

For the former, suppose that the constants map $c : A \rightarrow (A \rightarrow A)$ is an equivalence. Then we have $a_0 : A$ such that $c(a_0) = \text{id}_A$. Hence, for every $a : A$, we must have $a = \text{id}_A(a) = c(a_0)(a) = a_0$, so A is contractible.

For the latter, if A is contractible at $a_0 : A$, then any map $f : A \rightarrow X$ is fully determined by $f(a_0)$, i.e. a single point of X . ■

The universal property of the natural numbers and some consequences

The elimination rule of $\mathbb{N}$ allows us to define a function $f : \mathbb{N} \rightarrow X$ if we know where to map 0 and how to handle successors. In fact, given $x_0 : X$ and a map $f : X \rightarrow X$, there is a unique $u : \mathbb{N} \rightarrow X$ such that $u(0) = x_0$ and $u(n+1) = f(u(n))$ for every $n : \mathbb{N}$. Indeed, u can be defined recursively via the mentioned equations.

In homotopy type theory, “*there is a unique*” is formulated as saying that a Σ -type is contractible. What is

noteworthy is that, in the below, X can be any type, not necessarily a set, so that the equations are genuine data.

Universal property of the natural numbers. For every type X , point $x_0 : X$ and endomap $f : X \rightarrow X$, the type

$$\sum(u : \mathbb{N} \rightarrow X), (u(0) = x_0) \times \prod(n : \mathbb{N}), u(n+1) = f(u(n)) \quad (1)$$

is contractible.

Proof sketch. Given $u : \mathbb{N} \rightarrow X$, we can show (exercise!) that the type expressing the equations in (1) is equivalent to $\prod(n : \mathbb{N}), u(n) = r(n)$ where r is defined recursively by $r(0) = x_0$ and $r(n+1) = f(r(n))$. Hence, the type (1) is equivalent to the contractible type $\sum(u : \mathbb{N} \rightarrow X), u = r$, and hence itself contractible. ■

Corollary. The map from $\sum(f : \mathbb{N} \rightarrow X), (\prod(n : \mathbb{N}), f(n+1) = f(n))$ to X that evaluates f at 0 is an equivalence, for any type X .

Proof. We build a chain of equivalences that does the job.

$$\begin{aligned} & \sum(f : \mathbb{N} \rightarrow X), \left(\prod(n : \mathbb{N}), f(n+1) = f(n) \right) \\ \simeq & \sum(f : \mathbb{N} \rightarrow X), \underbrace{\left(\sum(x_0 : X), f(0) = x_0 \right)}_{\text{contractible!}} \times \left(\prod(n : \mathbb{N}), f(n+1) = f(n) \right) \\ \simeq & \sum(x_0 : X), \underbrace{\left(\sum(f : \mathbb{N} \rightarrow X), (f(0) = x_0) \times \left(\prod(n : \mathbb{N}), f(n+1) = f(n) \right) \right)}_{\text{contractible by (1) instantiated with the identity endomap}} \\ \simeq & X \end{aligned}$$

Similarly, since $\mathbb{Z}$ can be decomposed (or defined as) as $\mathbb{N} + \mathbb{1} + \mathbb{N}$ one shows:

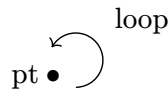
Corollary[eval-at-0-is-equiv]. The map from $\sum(f : \mathbb{Z} \rightarrow X), f \circ \text{succ} = f$ to X that evaluates at 0 is an equivalence. ■

2. The circle

So far, we have seen inductive types generated by (point) constructors, e.g. the natural numbers are generated by 0 and succ, with introduction rules yielding new points. What if we also allowed the introduction of new *identifications*?

This is the idea behind **higher inductive types (HITs)**. One example is the propositional truncation which we can realize as follows. For a type A , let $\|A\|$ be the type with constructor $|a| : \|A\|$ for all $a : A$, and add an identification $t = t'$ for all $t, t' : \|A\|$.

We now consider another example: **the circle** S^1 : generated by a single point pt and a single identification $\text{loop} : \text{pt} = \text{pt}$ of that point with itself, suggestively drawn below.



Formation $\overline{\Gamma \vdash S^1} \text{ Type}$

Introduction $\overline{\Gamma \vdash \text{pt} : S^1} \quad \overline{\Gamma \vdash \text{loop} : \text{pt} = \text{pt}}$

Following the general pattern of the other type formers, for the elimination and computation rules we want something like the following.

Elimination (preliminary)

$$\frac{\Gamma, x : S^1 \vdash A(x) : \text{Type} \quad \Gamma \vdash a : A[\text{pt}/x] \quad ??? \quad \Gamma \vdash s : S^1}{\Gamma \vdash S^1\text{-elim}(a, s) : A[s/x]}$$

Computation (preliminary)

$$\frac{\Gamma, x : S^1 \vdash A(x) : \text{Type} \quad \Gamma \vdash a : A[\text{pt}/x] \quad ???}{\Gamma \vdash S^1\text{-elim}(a, \text{pt}) \equiv a}$$

We would similarly expect a computation rule for applying S^1 -elim to loop. But simply writing $S^1\text{-elim}(a, \text{loop})$ does not make sense since loop is not of type S^1 .

Fortunately, functions can act on identifications (recall ap), we just need a version for *dependent* functions that we introduce now.

Recall that $\text{ap}_f : x = x' \rightarrow f(x) = f(x')$. Now for a dependent function $f : \prod(x : X), A(x)$, we can't directly compare $f(x)$ and $f(x')$ as they live in $A(x)$ and $A(x')$, respectively. But of course we can transport, so we make the following definition.

Definition (Dependent action on paths). Given a dependent function $f : \prod(x : X), A(x)$ and $x, x' : X$, the **dependent action** of f on p is the dependent function

$$\text{apd}_f : \prod(p : x = x'), \text{transport}^A(f(x), p) = f(x')$$

given by $\text{apd}_f \text{ refl} := \text{refl}$.

NB. For a nondependent function $f : X \rightarrow A$, we can relate apd_f and ap_f via the fact that transport in a nondependent type is trivial, i.e. $\text{transport}^{x \mapsto A}(a, p) = a$ for any $p : x_1 \overline{X} x_2$ and $a : A$ ([exercise!](#)).

This suggests the following rules that we indeed adopt, where the type of l is derived from that apd_f .

Elimination

$$\frac{\Gamma, x : S^1 \vdash A(x) : \text{Type} \quad \Gamma \vdash a : A[\text{pt}/x] \quad \Gamma \vdash l : \text{transport}^A(a, \text{loop}) = a \quad \Gamma \vdash s : S^1}{\Gamma \vdash S^1\text{-elim}(a, l, s) : A[s/x]}$$

Computation

$$\frac{\Gamma, x : S^1 \vdash A(x) : \text{Type} \quad \Gamma \vdash a : A[\text{pt}/x] \quad \Gamma \vdash l : \text{transport}^A(a, \text{loop}) = a}{\Gamma \vdash S^1\text{-elim}(a, l, \text{pt}) \equiv a}$$

$$\frac{\Gamma, x : S^1 \vdash A(x) : \text{Type} \quad \Gamma \vdash a : A[\text{pt}/x] \quad \Gamma \vdash l : \text{transport}^A(a, \text{loop}) = a}{\Gamma \vdash \text{apd}_{\lambda x. S^1\text{-elim}(a, l, x)}(\text{loop}) = l}$$

NB. We only require a non-definitional equality in the last rule, because the left-hand side is not a new primitive, but a composite term. On the other hand, note that the first computation rule must hold definitionally for the second rule to type-check as written.

Nondependent elimination for S^1 To define a (nondependent) function $f : S^1 \rightarrow A$, the elimination rule implies that it's sufficient to give $a : A$ and $l : a = a$ because transports in nondependent types are trivial (as remarked above).

This function will satisfy $f(\text{pt}) \equiv a$ and $\text{ap}_f(\text{loop}) = l$.

[Advanced note] It is not strictly necessary to add a new type, as long as we're willing to have non-definitional computation rules. Indeed, Bezem, Buchholtz and Grayson have shown that the type of $\mathbb{Z}$ -torsors

$$\sum(X : \mathcal{U}_0), \sum(f : X \rightarrow X), \|(Z, \text{succ}) = (X, f)\|$$

obeys the induction principle of the circle, cf. [doi:10.1016/j.jpaa.2021.106687](https://doi.org/10.1016/j.jpaa.2021.106687).

The circle and its universal properties

Using the elimination principle, along with some basic lemmas about transport, we can internalize the introduction/elimination rules and prove the (dependent) universal property of the circle ([exercise!](#)).

Universal property of the circle. The map

$$\begin{aligned} (S^1 \rightarrow A) &\longrightarrow \sum(a : A), a = a \\ f &\mapsto (f(\text{pt}), \text{ap}_f(\text{loop})) \end{aligned}$$

is an equivalence.

In short, a nondependent function $S^1 \rightarrow A$ is precisely given by a point $a : A$ and a loop $l : a = a$.

Dependent universal property of the circle. The map

$$\begin{aligned} \prod(s : S^1), A(s) &\longrightarrow \sum(a : A(\text{pt})), \text{transport}^A(a, \text{loop}) = a \\ f &\mapsto (f(\text{pt}), \text{apd}_f(\text{loop})) \end{aligned}$$

is an equivalence.

A propositional induction principle for the circle

Note that if we have a *proposition*-valued dependent type $P(x)$ over S^1 , then $\text{transport}^P(p_0, \text{loop}) = p_0$ always holds for any $p_0 : P(\text{pt})$, because $P(\text{pt})$ is a proposition.

Thus, for such proposition-valued P , to prove $\forall(s : S^1), P(s)$ it suffices to prove $P(\text{pt})$.

In particular, we see that $\|x = y\|$ holds for all $x, y : S^1$ (the circle is said to be **connected**), because it suffices to prove $\|\text{pt} = \text{pt}\|$ which we can do by `|refl|`.

However, we do *not* have $\prod(x, y : S^1), x = y$, because this would make the circle a proposition, forcing $\text{pt} = \text{pt}$ to be contractible; but we will show that it's equivalent to $\mathbb{Z}$.

This nicely illustrates the dramatic effect that the propositional truncation can have.

3. The fundamental group of the circle is given by the integers

The obvious strategy of showing that the map $w : \mathbb{Z} \rightarrow \text{pt} = \text{pt}$ by $k \mapsto \text{loop}^k$ is an equivalence is not really feasible. Since $\text{pt} = \text{pt}$ is an identity type, we turn to the fundamental theorem of identity types, picking $X := S^1$ and $x_0 := \text{pt}$.

We define $C : S^1 \rightarrow \mathcal{U}_0$ by the non-dependent elimination principle, so that $C(\text{pt}) := \mathbb{Z}$ and $\text{ap}_C(\text{loop}) := \text{eqtoid}(\text{succ}) : \mathbb{Z} = \mathbb{Z}$ where $\text{succ} : \mathbb{Z} \simeq \mathbb{Z}$ is the successor equivalence. We equip $C(\text{pt}) \equiv \mathbb{Z}$ with the point $c_0 := 0$.

We now have to show that the total space of C is contractible. We use the universal property of contractible types for this and show that the constants map $X \rightarrow ((\sum(s : S^1), C(s)) \rightarrow X)$ is an equivalence.

For the codomain of this map, we note that we have equivalences, with the second map given by the dependent universal property of the circle.

$$\begin{array}{ccc}
(\sum(s : S^1), C(s)) \rightarrow X & \xrightarrow[\lambda s. \lambda c. h(s, c)]{\simeq} & \prod(s : S^1)(C(s) \rightarrow X) \\
& & \downarrow \simeq \quad h \mapsto (h(\text{pt}), \text{apd}_h(\text{loop})) \\
& & \sum(f : C(\text{pt}) \rightarrow X), \text{transport}^{\lambda s : S^1. C(s) \rightarrow X}(f, \text{loop}) = f
\end{array}$$

Characterizing the transport

We take a moment to describe the transport. The recipe for such calculations is always the same: **generalize the statement to something that type-checks for general paths and prove it by path induction.**

In this case, we consider a type A (generalizing S^1) with a dependent type B (generalizing C) over it, a path (not *necessarily* a loop!) $p : a_1 = a_2$ in A (generalizing loop), and a function $f : B(a_1) \rightarrow X$.

Then it makes sense to consider $\text{transport}^{\lambda a. B(a) \rightarrow X}(f, p) : B(a_2) \rightarrow X$.

I claim that it must be equal to $B(a_2) \xrightarrow{\lambda b : B(b). \text{transport}^B(b, p^{-1})} B(a_1) \xrightarrow{f} X$.

Indeed, by path induction, we only need to consider $p \equiv \text{refl}$ and then it's clearly true.

Applying the characterization of the transport, we get an equivalence

$$\begin{array}{c}
\sum(f : C(\text{pt}) \rightarrow X), \text{transport}^{\lambda s : S^1. C(s) \rightarrow X}(f, \text{loop}) = f \\
\downarrow \simeq \\
\sum(f : C(\text{pt}) \rightarrow X), f \circ (\lambda c : C(\text{pt}). \text{transport}^C(c, \text{loop}^{-1})) = f
\end{array}$$

Characterizing another transport

General fact 1. For $B : A \rightarrow \mathcal{U}$ and a path $p : a_1 = a_2$ in A , the functions

$$\lambda b : B(a_1). \text{transport}^B(b, p) \quad \text{and} \quad \text{idtofun}(\text{ap}_B(p))$$

of type $B(a_1) \rightarrow B(a_2)$ are equal.

Proof. By path induction on p ; for $p \equiv \text{refl}$ both maps are just the identity.

General fact 2. A variation: the functions

$$\lambda b : B(a_1). \text{transport}^B(b, p^{-1}) \quad \text{and} \quad \text{idtofun}(\text{ap}_B(p))^{-1}$$

of type $B(a_2) \rightarrow B(a_1)$ are equal. (Here we use that idtofun always returns an equivalence.)

Proof. By path induction on p .

Applying general fact 2 to our main proof, we see that

$$\lambda c. \text{transport}^C(c, \text{loop}^{-1}) = \text{idtofun}(\text{ap}_C(\text{loop}))^{-1} = \text{idtofun}(\text{eqtoid}(\text{succ}))^{-1} = \text{succ}^{-1} \quad (2)$$

by our choice of C and by computing the composition $\text{idtofun} \circ \text{eqtoid}$.

Using this, we get

$$\begin{array}{c}
\Sigma(f : C(\text{pt}) \rightarrow X), f \circ (\lambda c : C(\text{pt}). \text{transport}^C(c, \text{loop}^{-1})) = f \\
\downarrow \simeq \\
\Sigma(f : C(\text{pt}) \rightarrow X), f \circ \text{succ}^{-1} = f \\
\downarrow \text{id} \\
\Sigma(f : \mathbb{Z} \rightarrow X), f \circ \text{succ}^{-1} = f \\
\downarrow \simeq \text{ev}_0 \quad \text{— cf. Corollary[eval-at-0-is-equiv]} \\
X
\end{array}$$

Putting everything together, we get the commutative diagram (Figure 1), which shows that the constants map must be an equivalence (by 3-for-2), as we wanted.

Indeed, we now know that the total space of C is contractible, so that $C(s)$ must be equivalent to $\text{pt} = s$ for every $s : S^1$. In particular, we can take $s \equiv \text{pt}$ to get that $C(\text{pt})$, which is defined to be $\mathbb{Z}$ and $\text{pt} = \text{pt}$ are equivalent. ■

[Advanced note] Some of the above could be reduced to descent for pushouts and the observation that S^1 is the pushout of $\mathbb{1} \leftarrow 2 \rightarrow \mathbb{1}$ (i.e. the suspension of the booleans). Roughly speaking, descent tells us that a Σ -type over a pushout is again a pushout (of Σ -types). This can be used to describe $\Sigma(s : S^1), C(s)$.

$$\begin{array}{ccc}
(\Sigma(s : S^1), C(s)) \rightarrow X & \xrightarrow[\quad h \mapsto \lambda s. \lambda c. h(s, c) \quad]{\simeq} & \prod(s : S^1)(C(s) \rightarrow X) \\
\uparrow x \mapsto \lambda(s, c).x & & \downarrow \simeq \quad h \mapsto (h(\text{pt}), \text{apd}_h(\text{loop})) \\
& & \Sigma(f : C(\text{pt}) \rightarrow X), \text{transport}^{\lambda s : S^1. C(s) \rightarrow X}(f, \text{loop}) = f \\
& & \downarrow \simeq \\
& & \Sigma(f : C(\text{pt}) \rightarrow X), f \circ (\lambda c : C(\text{pt}). \text{transport}^C(c, \text{loop}^{-1})) = f \\
& & \downarrow \simeq \\
& & \Sigma(f : C(\text{pt}) \rightarrow X), f \circ \text{succ}^{-1} = f \\
& & \downarrow \text{id} \\
& & \Sigma(f : \mathbb{Z} \rightarrow X), f \circ \text{succ}^{-1} = f \\
& & \downarrow \simeq \quad \text{ev}_0 \\
X & \xrightarrow[\quad \text{id} \quad]{} & X
\end{array}$$

Figure 1: Commutative diagram with all major components of our proof.

4. The equivalence is a group isomorphism

While we established an equivalence $\mathbb{Z} \rightarrow \text{pt} = \text{pt}$, we did not yet show that it acts as expected, i.e. that it sends $k : \mathbb{Z}$ to loop^k . In particular, it would follow that our equivalence is a group homomorphism (and hence an isomorphism), since it would send $k + m$ to the path concatenation loop^{k+m} of loop^k and loop^m .

Proving this follows fairly directly from the following general fact.

Lemma (Transports are natural). Let X be a type with two type families A and B over it. Suppose we have $\psi : \prod (x : X), A(x) \rightarrow B(x)$. Then for any $p : x_1 = x_2$, the diagram

$$\begin{array}{ccc} A(x_1) & \xrightarrow{\psi_{x_1}} & B(x_1) \\ \text{transport}^A(-, p) \downarrow & & \downarrow \text{transport}^B(-, p) \\ A(x_2) & \xrightarrow{\psi_{x_2}} & B(x_2) \end{array}$$

commutes.

Proof. By path induction (what else?).

Specializing $A(x)$ to $x_0 = x$ for some fixed $x_0 : X$ and calculating its transport (with path induction), we get the following instance that we can apply in our situation.

Corollary. The diagram

$$\begin{array}{ccc} x_0 = x_1 & \xrightarrow{\psi_{x_1}} & B(x_1) \\ (-) \bullet p \downarrow & & \downarrow \text{transport}^B(-, p) \\ x_0 = x_2 & \xrightarrow{\psi_{x_2}} & B(x_2) \end{array}$$

commutes. ■

Applying this to our maps $\varphi_s : \text{pt} = s \rightarrow C(s)$ with $s \equiv \text{pt}$ and $p \equiv \text{loop}$, we get, writing just φ for φ_{pt} , that

$$\text{transport}^C(\varphi(q), \text{loop}) = \varphi(q \bullet \text{loop}) \quad (3)$$

for any $q : \text{pt} = \text{pt}$.

Taking $q \equiv \text{loop}^k$ in (3), for $k : \mathbb{Z}$, we get

$$\text{succ}(\varphi(\text{loop}^k)) = \text{transport}^C(\varphi(\text{loop}^k), \text{loop}) = \varphi(\text{loop}^k \bullet \text{loop}) = \varphi(\text{loop}^{k+1}),$$

where the first equation is proved similarly to (2).

Note that $\varphi(\text{refl}) \equiv 0$, because 0 was our choice of base point for $\mathbb{Z} \equiv C(\text{pt})$ in the application of the fundamental theorem of identity types. Therefore, it now follows by induction on k that

$$\varphi(\text{loop}^k) = k.$$